Tutorial Sheet 0
Getting started

Fred Hamprecht
Simon Wagner

This exercise sheet introduces you to the main tools you will need for the exercises this semester:

- Git – the most widely used version control system, which we use to distribute and update course materials.
- VS Code – one of the most popular code editors, with tools for writing, running, and debugging programs.
- AI coding agents – increasingly part of the standard toolkit for modern software development, capable of writing, running, and debugging code from natural-language instructions.

The sheet is intended to be solved in the tutorial, not at home.

Git Setup

In this course, we use Git to distribute exercise sheets and accompanying materials, such as code and datasets. Install Git for your operating system. If you are new to Git, see the official Git tutorial. You only need to **clone our course repository** once. In a terminal (*Git Bash* on Windows), run:

```
git clone https://github.com/sciai-lab/mlph-26w.git
cd mlph-26w
```

Before each new sheet, run `git pull` *inside* `mlph-26w` to download and integrate updates. Keep the original templates unchanged and store your own work in a separate folder, e.g. `mlph-26w/my-solutions`, so that `git pull` does not run into conflicts with your local changes. The repository's `.gitignore` already excludes this folder and the virtual environment created below.

VS Code Setup

VS Code is a free code editor with an integrated terminal, Git support, and extensions for languages such as Python. We recommend using VS Code for the exercises in this course. You are free to use another IDE or code editor if you prefer. Note, however, that we may not be able to help with problems specific to other editors.

- Download VS Code from code.visualstudio.com/download and install it.
- Install Python 3 if it is not already installed.
- Within VS Code, **open the cloned `mlph-26w` folder** via **File** → **Open Folder**.
- In Extensions, install Microsoft's **Python** extension.
- Open the Command Palette (**Ctrl+Shift+P**, or **Cmd+Shift+P** on macOS) and run **Python: Create Environment** to create a **venv**. Install packages into this environment whenever an exercise requires them.

AI Coding Agents

Several exercises throughout the semester will involve the use of AI coding agents to familiarize you with modern agentic coding workflows. Unlike a chatbot, an agent works directly on your computer: it can read and edit files, run code, and check the results.

See the **first tutorial's introductory slides** for an overview of agent options and approximate costs.

Choose an agent, then set it up in one of two ways:

- **Extension:** install the official VS Code extension for your chosen agent, open its panel, and sign in.
- **CLI:** follow the installation guide on the official website of your chosen agent. In VS Code's terminal, inside `mlph-26w`, launch the respective CLI and sign in.

Exercise: Agent-assisted physics data exploration

This exercise showcases a small data analysis pipeline in which an AI coding agent helps you select, acquire, load, and analyze a physics dataset.

Use the agent for (a) and (b). Do (c) yourself, without the agent.

(a) Select and acquire data. Select a publicly downloadable physics dataset that interests you—for example, from accelerator experiments, cosmic-ray measurements, astronomical observations, or condensed-matter physics. It must contain categorical class labels with a reasonable number of classes and at least **10 numerical features**, excluding labels and identifiers, and occupy at most **100 MB** after decompression. Write code to download and load it. Report the source; the physical meaning of the observations, features, and labels; the number of observations and features; and the class counts.

(b) Visualize with PCA. Select suitable numerical features (excluding class labels and identifiers), handle missing or invalid values, and apply appropriate preprocessing (e.g. standardization, since PCA is sensitive to feature scales). Report the selected features and preprocessing steps. Produce a two-dimensional PCA projection: first plot it without class colors, then color the same points by class and include a legend.

(c) Check and interpret. Verify the agent's claims from (a) and (b), such as feature meanings, units, and label definitions, against the source documentation, and note anything the agent got wrong. Briefly explain whether the plots show distinct groups and how these relate to the classes, and name one limitation of the analysis.